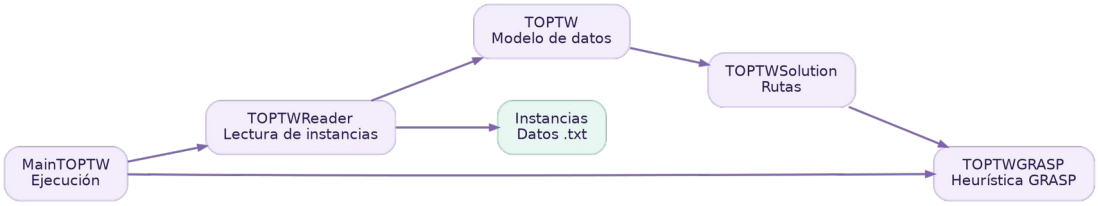
INFORME TÉCNICO / PRÁCTICA 03

Calidad del producto software y documentación

SonarQube · Maven · Doxygen · MkDocs



RTVRPTW · Recomendador turístico con ventanas de tiempo (ExpositoTOP)

GRADO EN INGENIERÍA INFORMÁTICA · UNIVERSIDAD DE LA LAGUNA

Asignatura: Laboratorio de Desarrollo y Herramientas

Autor: Daniel Alejandro Álvarez Casablanca

Curso académico: 2026–2027

Estado del trabajo: Maven y SonarQube verificados en Windows/Cursor el 01/11/2026; correcciones, documentación y publicación pendientes de validar.

Índice de contenidos

01 Resumen y objetivos....3

02 Arquitectura y migración a Maven....3

03 Ciclo de vida y pruebas de construcción....4

04 Plugins configurados....4

05 Análisis de calidad con SonarQube....5

06 Problemas, soluciones y reflexión crítica....6

07 Documentación automática: Doxygen y Graphviz....6

08 Portal Docs-as-Code: MkDocs, Material y LLM....6

09 Entregables, validación y publicación....7

10 Conclusiones y fuentes....8

Anexo · Matriz de cumplimiento....9
Anexo II · Evidencias de Maven en Windows....10
Anexo III · Análisis inicial de SonarQube....13

01 Resumen y objetivos

La práctica consiste en analizar y mejorar la calidad y la documentación del proyecto Java RTVRPTW, distribuido como ExpositoTOP.rar. Se ha reorganizado el código conforme a Maven, creado un conjunto inicial de pruebas y configurado la cadena de documentación que conecta Doxygen, Graphviz y MkDocs. El trabajo respeta el enfoque Docs-as-Code propuesto en el guion.

Criterio de evidencia

El informe diferencia resultados verificados mediante capturas del estudiante de configuraciones todavía no ejecutadas. El bloque Maven está acreditado en Windows; SonarQube, Doxygen y MkDocs permanecen sujetos a validación en el equipo.

Componente	Estado al 01/11/2026	Evidencia
Estructura Maven	Verificada	pom.xml y carpetas convencionales
Compilación Java	Verificada en Maven	11 fuentes Java; BUILD SUCCESS
Ejecución de muestra	Comprobada	Instancia c101Recortada, modo --quick
Pruebas de verificación directa	Comprobadas	6 comprobaciones directas en preparación
Maven + JUnit + JaCoCo	Verificado en Windows	6 fases BUILD SUCCESS; JaCoCo generó informe
SonarQube / Doxygen / MkDocs	SonarQube analizado	5 issues; 12,6 % cobertura; docs pendientes

02 Arquitectura y migración a Maven

La carpeta original incluía build.xml y configuración NetBeans/Eclipse, no una estructura Maven estándar. Se han conservado los paquetes top, es.ull.esit.utilities y es.ull.esit.utils, reubicado las once clases Java y empaquetado las instancias como recursos para que funcionen también desde el JAR. Las dependencias del código principal son las del propio JDK.

Ruta nueva	Contenido
src/main/java	11 clases Java del proyecto adaptadas
src/main/resources/Instances	Datos de las instancias TOPTW
src/test/java	7 pruebas JUnit 5 distribuidas en 2 clases
pom.xml	Metadatos, Java 17, plugins y despliegue local
Doxyfile + mkdocs.yml	Configuración de documentación y portal

Modelo funcional

MainTOPTW selecciona la instancia; TOPTWReader carga los datos; TOPTW representa puntos, ventanas y distancias; TOPTWSolution conserva el estado de las rutas; y TOPTWGRASP construye soluciones candidatas mediante listas restringidas de candidatos. TOPTWEvaluator se encuentra incompleto en el material de origen, por lo que no se presenta como una capacidad disponible.

03 Ciclo de vida de Maven

El ciclo de vida Maven es secuencial: cada fase ejecuta las anteriores necesarias. En Windows se verificaron validate, compile, test, package, install y deploy con BUILD SUCCESS. El despliegue usa un repositorio local de demostración, no Maven Central.

Fase	Comando	Resultado comprobado
validate	mvn validate	BUILD SUCCESS; POM validado
compile	mvn compile	BUILD SUCCESS; 11 fuentes Java compiladas
test	mvn test	BUILD SUCCESS; 7/7 pruebas sin fallos
package	mvn package	BUILD SUCCESS; JAR generado
install	mvn install	BUILD SUCCESS; instalado en .m2
deploy	mvn deploy	BUILD SUCCESS; repositorio local de práctica

Ejecución desde Cursor

Las seis fases se ejecutaron desde el panel Maven > Lifecycle de Cursor (Windows, PowerShell), con salida BUILD SUCCESS. Además se comprobó mvn clean verify en terminal. Las capturas originales permiten acreditar la ejecución desde el IDE y la línea de comandos.

```
mvn clean verify   # BUILD SUCCESS; 7 pruebas superadas
java -jar target/expositotop-1.0.0-SNAPSHOT.jar --quick
```

Pruebas y verificación

La captura de mvn test muestra 7 pruebas JUnit 5 superadas en dos clases: ExpositoUtilitiesTest (3) y TOPTWReaderTest (4), con 0 fallos, 0 errores y 0 pruebas omitidas. Maven compiló 11 archivos Java y package generó el JAR. En install, JaCoCo procesó datos de 13 clases, sin que esa cifra represente un porcentaje de cobertura.

04 Plugins Maven configurados

Plugin	Responsabilidad	Contribución
Compiler	Compilar con Java 17	Homogeneiza la compilación
Surefire	Ejecutar JUnit 5	Evita regresiones básicas
JAR	Empaquetar con MainTOPTW	Permite java -jar
Exec	Ejecutar la clase principal	Facilita pruebas rápidas desde IDE
JaCoCo	Generar cobertura XML	Métrica de pruebas para SonarQube
SonarScanner	Analizar código en servidor	Integra análisis con Maven

La centralización en el POM facilita repetir el mismo proceso desde terminal, IDE o CI y reduce el trabajo manual respecto a una construcción aislada de Ant.

05 Análisis de calidad con SonarQube

Estrategia de análisis

La integración se ha configurado mediante SonarScanner for Maven, la URL http://localhost:9000 y el identificador expositotop-practica3. El análisis debe ejecutarse después de compilar y verificar las pruebas, de manera que SonarQube pueda acceder al bytecode y al XML de cobertura de JaCoCo.

```
$env:SONAR_TOKEN="TOKEN_PRIVADO"
.\scripts\sonar.ps1
Remove-Item Env:SONAR_TOKEN
```

Análisis real ejecutado el 01/11/2026 con SonarScanner for Maven en Windows 11 y servidor SonarQube local. El Quality Gate fue PASSED. El panel mostró 5 incidencias abiertas de fiabilidad, 0 incidencias de seguridad, 0,0 % de duplicación y 12,6 % de cobertura. El token utilizado es secreto y no se incorpora al informe.

Resultados de la primera medición (antes de las correcciones)

Métrica solicitada	Resultado a incorporar desde SonarQube
Quality Gate	PASSED; todas las condiciones del perfil utilizado superadas.
Incidencias abiertas	5: fiabilidad (1 Blocker, 1 High, 3 Medium).
Seguridad / mantenibilidad	0 incidencias de seguridad y 0 de mantenibilidad en el panel Issues.
Duplicación / esfuerzo	0,0 % de duplicación; 35 min de esfuerzo estimado para los 5 avisos.
Cobertura de pruebas	12,6 %; JaCoCo importado mediante Maven.
Comparación tras correcciones	PENDIENTE: repetir SonarQube; no existe una medición final acreditada.

Lectura crítica de las métricas

La puerta de calidad supera las condiciones configuradas, aunque siguen presentes cinco problemas de fiabilidad: uno bloqueante, uno alto y tres medios. La ausencia de duplicación detectada no sustituye a pruebas funcionales. La cobertura del 12,6 % exige ampliar los escenarios JUnit, sobre todo las rutas alternativas y los casos límite.

06 Revisión de problemas y propuestas

Las cinco incidencias siguientes proceden del panel Issues real de SonarQube (01/11/2026). La revisión identifica su causa y propone cambios concretos. Se preparó una versión corregida del código que deberá compilarse, probarse y analizarse nuevamente desde Windows; aún no se dispone de segunda medición.

Problema observado	Actuación	Estado
ExpositoUtilities.java:82 · Blocker	Cerrar BufferedWriter con try-with-resources	Corrección preparada
TOPTWGRASP.java:78 · High	Reutilizar un campo Random en lugar de crear uno por invocación	Corrección preparada
PowerSet.java:56 · Medium	Devolver un Iterator independiente en cada recorrido	Corrección + prueba preparadas
TOPTWReader.java:39 · Medium	Comprobar que la línea de cabecera existe	Corrección preparada
TOPTWSolution.java:62 · Medium	Renombrar equals(TOPTWSolution) a hasSamePredecessors	Corrección preparada
Cobertura general · 12,6 %	Añadir pruebas de rutas, errores y casos límite	Pendiente

El objetivo del segundo análisis será comprobar si disminuyen las incidencias, sin declarar solucionados los avisos hasta contrastar un nuevo panel. La mejora prioritaria adicional es aumentar la cobertura por encima del 12,6 %.

07 Documentación con Doxygen y Graphviz

Se han incorporado comentarios compatibles con Doxygen, incluyendo etiquetas \brief para clases y métodos fundamentales y explicaciones detalladas de la lectura de instancias. El Doxyfile recorre recursivamente los archivos Java, activa la generación HTML y LaTeX, y habilita Graphviz con HAVE_DOT = YES y DOT_IMAGE_FORMAT = svg.

```
New-Item -ItemType Directory -Force .artifacts | Out-Null
doxygen Doxyfile
Start-Process .\.artifacts\doxygen\html\index.html
```

El proyecto incluye el diagrama de arquitectura y la configuración para generar grafos automáticos en Doxygen. Todavía se debe ejecutar Doxygen en Windows y aportar capturas de la salida HTML y de los diagramas Graphviz.

08 Portal Docs-as-Code con MkDocs

El archivo mkdocs.yml configura MkDocs con tema Material, búsqueda y MkDoxy para extraer la API Java desde los comentarios Doxygen. La organización incluye introducción, arquitectura, manual de usuario, Maven, calidad de código, Doxygen y dos registros de decisiones (ADR). El plugin with-pdf se ha declarado para producir una documentación técnica PDF distinta de este informe.

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-docs.txt
.\scripts\docs.ps1
```

La salida prevista es site/index.html para el portal y site/pdf/documentacion-tecnica.pdf para la documentación técnica exportada. Estos ficheros son el resultado de la compilación de MkDocs y deberán generarse con acceso a sus dependencias. No se presentan como archivos ya exportados mediante esa herramienta.

Enriquecimiento mediante LLM

El manual, las guías operativas, las páginas de arquitectura y los ADR se redactaron con ayuda de un modelo de lenguaje y se contrastaron con el código extraído. Esta asistencia acelera la redacción, pero exige supervisión humana: la IA no debe inventar métricas, atribuir funcionalidades no implementadas o reproducir sin revisión diagnósticos de SonarQube.

09 Entrega y publicación

El repositorio GitHub debe incluir el código corregido, pom.xml, Doxyfile, mkdocs.yml, docs/ y el PDF técnico generado. La publicación opcional puede hacerse con MkDocs en GitHub Pages. El repositorio todavía no se ha creado desde este entorno; añade su URL real en el espacio siguiente tras publicar.

URL de GitHub: _____

Documento técnico PDF generado en Windows: _____

10 Conclusiones y reflexión

La práctica muestra la complementariedad entre construcción reproducible, pruebas, análisis estático y documentación. Maven aporta convenciones y un ciclo de vida común; SonarQube agrega señales de calidad; Doxygen formaliza la relación entre el código y su documentación; y MkDocs reúne explicaciones, referencias de API y decisiones técnicas en un portal versionable.

La aportación más importante en esta fase es pasar de un proyecto ligado a Ant/NetBeans a una base Maven reproducible con clases, recursos y pruebas organizados. También se han eliminado formas de gestión de errores que complicaban la automatización. En cambio, persisten limitaciones funcionales del código original: el evaluador y la búsqueda local no están completados. Se trata de trabajo real por acometer, no de defectos supuestamente arreglados.

Maven está acreditado mediante seis fases BUILD SUCCESS y siete pruebas JUnit superadas. SonarQube ya ofrece una línea base verificable: Quality Gate PASSED, cinco incidencias de fiabilidad y cobertura del 12,6 %. Se han preparado correcciones para los cinco avisos, pero falta ejecutar las pruebas y repetir el análisis en Windows para verificar el resultado. Doxygen, MkDocs, PDF técnico y repositorio GitHub siguen pendientes.

Estado de cierre

Requisito del guion	Entregado / preparado	Paso final del estudiante
1–3: estudio y órdenes de Maven	Guía y comandos; Maven instalado	Añadir salida de mvn -version al informe
4–6: Maven, POM y plugins	Maven y JUnit verificados	6 fases SUCCESS; 7/7 pruebas; capturas disponibles
7–8: SonarQube y mejoras	Análisis real: 5 issues; Quality Gate aprobado; correcciones preparadas	Reanalizar el código corregido; añadir comparación antes/después
9.1: Doxygen y Graphviz	Código y Doxyfile	Generar HTML y grafos
9.2–9.4: Docs-as-Code y PDF	docs/, mkdocs.yml, scripts	Construir portal y PDF con MkDocs
Entrega: GitHub + Campus Virtual	ZIP e informe inicial	Publicar, enlazar y entregar

Fuentes y documentación consultada

• Apache Maven, “Introduction to the Build Lifecycle”: https://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html

• Apache Maven, “POM Reference”: https://maven.apache.org/pom.html

• SonarSource, “SonarScanner for Maven”: https://docs.sonarsource.com/sonarqube-server/analyzing-source-code/scanners/sonarscanner-for-maven

• Doxygen, “Configuration”: https://www.doxygen.nl/manual/config.html

• MkDocs, “Configuration”: https://www.mkdocs.org/user-guide/configuration/

• MkDoxy en PyPI: https://pypi.org/project/mkdoxy/

• Material for MkDocs: https://squidfunk.github.io/mkdocs-material/

• MkDocs with PDF: https://github.com/orzih/mkdocs-with-pdf

Base documental de la práctica: “Práctica 3. Herramientas de Calidad del Producto Software y Documentación”, 3 páginas; código fuente e instancias ExpositoTOP.rar entregados en el aula virtual.

Anexo · Matriz de cumplimiento y evidencias

La matriz distingue resultados reales obtenidos el 01/11/2026 de configuraciones y cambios preparados. Las capturas Maven se recogen en el Anexo II y las de SonarQube en el Anexo III.

N.º	Actividad	Estado	Prueba pendiente
1	Estudio Maven	Documentado	Explicar POM, carpetas y ciclo de vida.
2	Instalación Maven	Verificado en Windows	Recomendable capturar mvn -version y java -version.
3	Órdenes terminal / IDE	Capturas IDE y terminal	6 fases desde Cursor; clean verify desde terminal.
4	Migración ExpositoTOP	Compilación y JAR verificados	Opcional: captura de java -jar con modo --quick.
5	Fases Maven	6 de 6 BUILD SUCCESS	Capturas del 01/11/2026; deploy local acreditado.
6	Plugins Maven	JUnit, Surefire y JaCoCo comprobados	7 pruebas; 0 errores; JaCoCo analizó 13 clases.
7	SonarQube	Analizado en SonarQube	PASSED; 5 issues; 12,6 % cobertura; evidencia 01/11/2026
8	Problemas y propuestas	5 incidencias identificadas y soluciones preparadas	Compilar, reanalizar y capturar la mejora real
9	Doxygen + MkDocs + LLM	Archivos preparados	HTML, grafos, sitio web, PDF técnico, capturas y revisión.

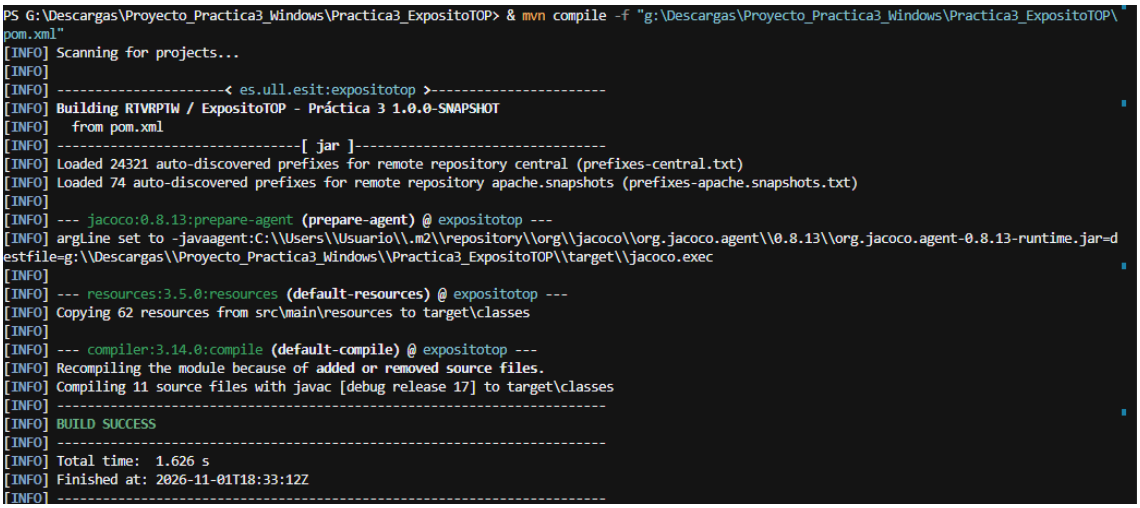
Pendiente de cierre: ejecutar las correcciones Java, repetir el análisis SonarQube y registrar nuevas métricas; generar HTML y diagramas Doxygen, el portal y PDF técnico de MkDocs y publicar el repositorio GitHub.

Anexo II · Evidencias de Maven en Windows

Ejecuciones realizadas el 1 de noviembre de 2026 desde Cursor y PowerShell. Se conservaron las capturas suministradas por el estudiante; los valores de esta sección proceden de dichas ejecuciones.

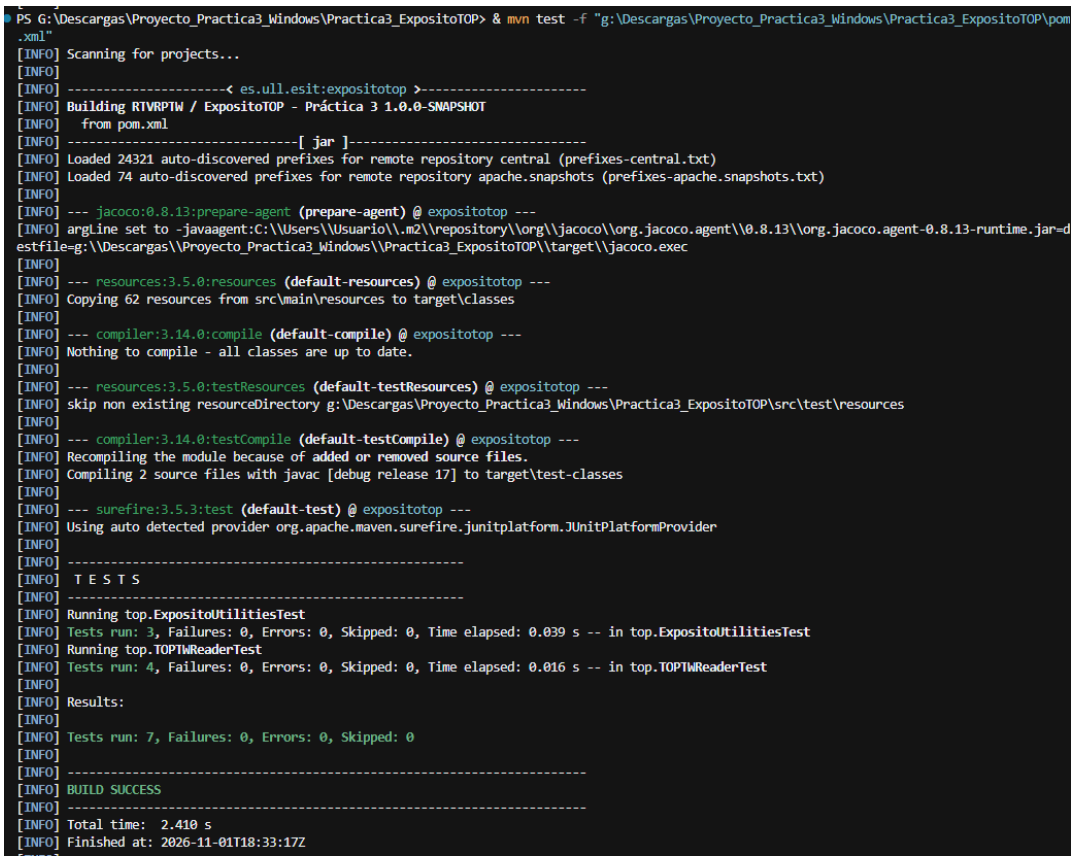
A. Compilación y pruebas automatizadas

Compilación: 11 archivos fuente Java; resultado BUILD SUCCESS.



```
PS G:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP> & mvn compile -f "g:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP\pom.xml"
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------------< es.ull.esit:expositotop >-----------------------
[INFO] Building RTVRPTW / ExpositoTOP - Práctica 3 1.0.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO] Loaded 24321 auto-discovered prefixes for remote repository central (prefixes-central.txt)
[INFO] Loaded 74 auto-discovered prefixes for remote repository apache.snapshots (prefixes-apache.snapshots.txt)
[INFO]
[INFO] --- jacoco:0.8.13:prepare-agent (prepare-agent) @ expositotop ---
[INFO] argLine set to -javaagent:C:\\Users\\Usuario\\.m2\\repository\\org\\jacoco\\org.jacoco.agent\\0.8.13\\org.jacoco.agent-0.8.13-runtime.jar=destfile=g:\\Descargas\\Proyecto_Practica3_Windows\\Practica3_ExpositoTOP\\target\\jacoco.exec
[INFO]
[INFO] --- resources:3.5.0:resources (default-resources) @ expositotop ---
[INFO] Copying 62 resources from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.14.0:compile (default-compile) @ expositotop ---
[INFO] Recompiling the module because of added or removed source files.
[INFO] Compiling 11 source files with javac [debug release 17] to target\classes
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.626 s
[INFO] Finished at: 2026-11-01T18:33:12Z
[INFO] ------------------------------------------------------------------------
```

Pruebas JUnit 5: 7 ejecutadas, 0 fallos, 0 errores y 0 omitidas; BUILD SUCCESS.



```
PS G:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP> & mvn test -f "g:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP\pom.xml"
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------------< es.ull.esit:expositotop >-----------------------
[INFO] Building RTVRPTW / ExpositoTOP - Práctica 3 1.0.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO] Loaded 24321 auto-discovered prefixes for remote repository central (prefixes-central.txt)
[INFO] Loaded 74 auto-discovered prefixes for remote repository apache.snapshots (prefixes-apache.snapshots.txt)
[INFO]
[INFO] --- jacoco:0.8.13:prepare-agent (prepare-agent) @ expositotop ---
[INFO] argLine set to -javaagent:C:\\Users\\Usuario\\.m2\\repository\\org\\jacoco\\org.jacoco.agent\\0.8.13\\org.jacoco.agent-0.8.13-runtime.jar=destfile=g:\\Descargas\\Proyecto_Practica3_Windows\\Practica3_ExpositoTOP\\target\\jacoco.exec
[INFO]
[INFO] --- resources:3.5.0:resources (default-resources) @ expositotop ---
[INFO] Copying 62 resources from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.14.0:compile (default-compile) @ expositotop ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.5.0:testResources (default-testResources) @ expositotop ---
[INFO] skip non existing resourceDirectory g:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP\src\test\resources
[INFO]
[INFO] --- compiler:3.14.0:testCompile (default-testCompile) @ expositotop ---
[INFO] Recompiling the module because of added or removed source files.
[INFO] Compiling 2 source files with javac [debug release 17] to target\test-classes
[INFO]
[INFO] --- surefire:3.5.3:test (default-test) @ expositotop ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running top.ExpositoUtilitiesTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.039 s -- in top.ExpositoUtilitiesTest
[INFO] Running top.TOPTWReaderTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.016 s -- in top.TOPTWReaderTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.410 s
[INFO] Finished at: 2026-11-01T18:33:17Z
```

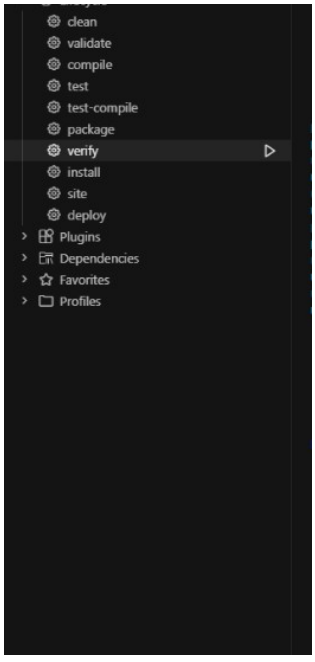
B. Empaquetado, despliegue e integración con Cursor

Las fases package, install y deploy terminaron correctamente. El artefacto se distribuyó al repositorio de la práctica denominado local-practica-snapshots (file://), sin publicarlo en ningún servicio externo.

Salida de deploy con transferencia al repositorio de práctica y BUILD SUCCESS.

```
PS G:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP> & mvn deploy -f "g:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP\pom.xml"
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------------< es.ull.esit:expositotop >-----------------------
[INFO] Building RTVRPTW / ExpositoTOP - Práctica 3 1.0.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO] Loaded 24321 auto-discovered prefixes for remote repository central (prefixes-central.txt)
[INFO] Loaded 74 auto-discovered prefixes for remote repository apache.snapshots (prefixes-apache.snapshots.txt)
[INFO]
[INFO] --- jacoco:0.8.13:prepare-agent (prepare-agent) @ expositotop ---
[INFO] argLine set to -javaagent:C:\\Users\\Usuario\\.m2\\repository\\org\\jacoco\\org.jacoco.agent\\0.8.13\\org.jacoco.agent-0.8.13-runtime.jar=destfile=g:\\Descargas\\Proyecto_Practica3_Windows\\Practica3_ExpositoTOP\\target\\jacoco.exec
[INFO]
[INFO] --- resources:3.5.0:resources (default-resources) @ expositotop ---
[INFO] Copying 62 resources from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.14.0:compile (default-compile) @ expositotop ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.5.0:testResources (default-testResources) @ expositotop ---
[INFO] skip non existing resourceDirectory g:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP\src\test\resources
[INFO]
[INFO] --- compiler:3.14.0:testCompile (default-testCompile) @ expositotop ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- surefire:3.5.3:test (default-test) @ expositotop ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Analyzed bundle 'RTVRPTW / ExpositoTOP - Práctica 3' with 13 classes
[INFO]
[INFO] --- install:3.2.0:install (default-install) @ expositotop ---
[INFO] Installing g:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP\pom.xml to C:\Users\Usuario\.m2\repository\es\ull\esit\expositotop\1.0.0-SNAPSHOT\expositotop-1.0.0-SNAPSHOT.pom
[INFO] Installing g:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP\target\expositotop-1.0.0-SNAPSHOT.jar to C:\Users\Usuario\.m2\repository\es\ull\esit\expositotop\1.0.0-SNAPSHOT\expositotop-1.0.0-SNAPSHOT.jar
[INFO]
[INFO] --- deploy:3.2.0:deploy (default-deploy) @ expositotop ---
Downloading from local-practica-snapshots: file://g:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP/.artifacts/maven-repository/es/ull/esit/expositotop/1.0.0-SNAPSHOT/maven-metadata.xml
Downloaded from local-practica-snapshots: file://g:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP/.artifacts/maven-repository/es/ull/esit/expositotop/1.0.0-SNAPSHOT/maven-metadata.xml (772 B at 86 kB/s)
Uploading to local-practica-snapshots: file://g:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP/.artifacts/maven-repository/es/ull/esit/expositotop/1.0.0-SNAPSHOT/expositotop-1.0.0-20261101.183617-2.pom
Uploaded to local-practica-snapshots: file://g:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP/.artifacts/maven-repository/es/ull/esit/expositotop/1.0.0-SNAPSHOT/expositotop-1.0.0-20261101.183617-2.pom (3.5 kB at 347 kB/s)
Uploading to local-practica-snapshots: file://g:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP/.artifacts/maven-repository/es/ull/esit/expositotop/1.0.0-SNAPSHOT/expositotop-1.0.0-20261101.183617-2.jar
Uploaded to local-practica-snapshots: file://g:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP/.artifacts/maven-repository/es/ull/esit/expositotop/1.0.0-SNAPSHOT/expositotop-1.0.0-20261101.183617-2.jar (161 kB at 16 MB/s)
Downloading from local-practica-snapshots: file://g:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP/.artifacts/maven-repository/es/ull/esit/expositotop/maven-metadata.xml
Downloaded from local-practica-snapshots: file://g:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP/.artifacts/maven-repository/es/ull/esit/expositotop/maven-metadata.xml (282 B at 94 kB/s)
Uploading to local-practica-snapshots: file://g:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP/.artifacts/maven-repository/es/ull/esit/expositotop/1.0.0-SNAPSHOT/maven-metadata.xml
Uploaded to local-practica-snapshots: file://g:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP/.artifacts/maven-repository/es/ull/esit/expositotop/1.0.0-SNAPSHOT/maven-metadata.xml (772 B at 154 kB/s)
Uploading to local-practica-snapshots: file://g:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP/.artifacts/maven-repository/es/ull/esit/expositotop/maven-metadata.xml
Uploaded to local-practica-snapshots: file://g:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP/.artifacts/maven-repository/es/ull/esit/expositotop/maven-metadata.xml (282 B at 94 kB/s)
[INFO] Skipping already-deployed es.ull.esit:expositotop:1.0.0-SNAPSHOT
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.727 s
[INFO] Finished at: 2026-11-01T18:36:20Z
[INFO] ------------------------------------------------------------------------
PS G:\Descargas\Proyecto_Practica3_Windows\Practica3_ExpositoTOP>
```

Vista Maven > Lifecycle en Cursor, desde la que se lanzaron las fases.

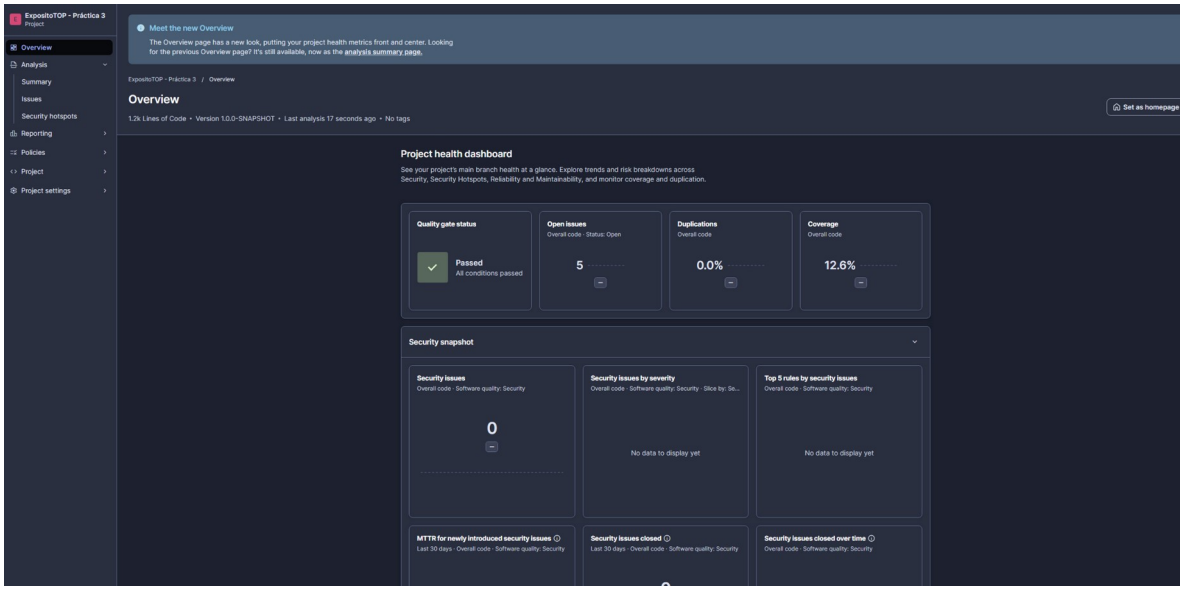


Verificaciones documentadas: validate, compile, test, package, install y deploy desde el panel de Cursor. La ejecución de deploy finaliza con BUILD SUCCESS. El repositorio Maven de despliegue es local; su éxito no implica publicación en Maven Central.

Anexo III · Evidencias del análisis SonarQube

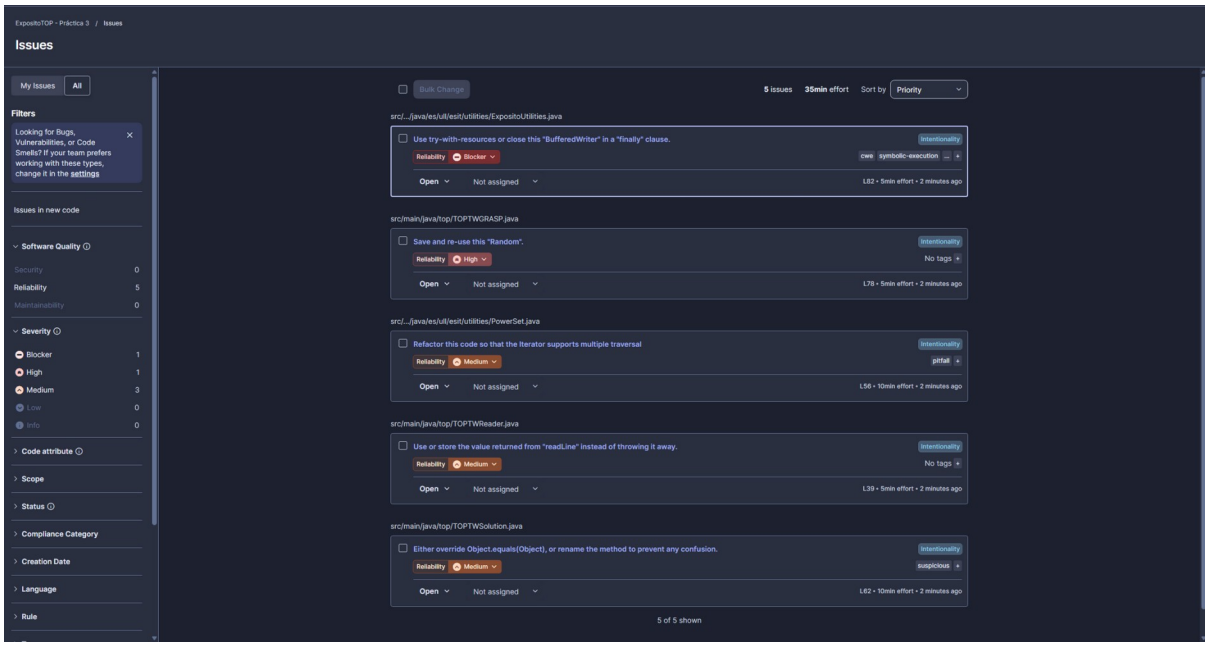
Capturas del análisis inicial efectuado el 1 de noviembre de 2026 desde SonarQube Community Build. Los resultados proceden del servidor local del estudiante; las correcciones aún no se han reanalizado.

A. Dashboard de calidad y cobertura



Quality Gate aprobado; 5 incidencias abiertas; duplicación del 0,0 % y cobertura de pruebas del 12,6 %.

B. Incidencias detectadas por SonarQube



Cinco hallazgos de fiabilidad: 1 bloqueante, 1 alto y 3 medios. Esfuerzo total estimado: 35 min.

Interpretación: las reglas señalan oportunidades de mejorar gestión de recursos, aleatoriedad, iteradores, lectura de ficheros y semántica de comparación. Para cerrar el ciclo de mejora es necesario repetir el escáner sobre la versión corregida y documentar el resultado.